

4.3 Best Practices

During refactoring, you may find yourself losing the overview and straying from your actual plan because you keep finding new places in the code that you want to improve. Here are some best practices to help you focus on the essentials.

4.3.1 Only the Internal Structure May Be Changed

When refactoring, you should concentrate on modifying only the internal structure of a function, but not its parameter list or return value. If you ask an AI tool for a refactoring, it will usually adhere to this recommendation. To be on the safe side, you can also specify in your prompt that the signature of the method shouldn't be changed.

Prompt: Don't change the signature of the method; only focus on the inner structure.

This ensures that the refactoring doesn't affect the rest of your application and takes effect only in separate individual places.

4.3.2 Small-Step Approach

Even if you only focus on the internal structure of a method or class, refactoring can take on larger proportions, depending on the role the respective structure plays in your application. To prevent refactoring from getting out of hand, you should try to proceed in as small steps as possible.

If you use AI tools such as GitHub Copilot or ChatGPT, you should first analyze your code and see what refactoring potential the class or function has. Then, you can address the individual points step by step. This procedure has the advantage that you can commit the individual steps to the version system. Thus, if an error occurs, you're at most one step away from a func-

tioning implementation and can repeat the last step using a modified prompt.

If the refactoring steps are too large and too many aspects are refactored at once, the risk of errors and undesirable behavior is significantly greater.

4.3.3 Test-Driven Refactoring

Before starting a refactoring process, you want to make sure that the class or function you intend to improve has been validated by automated tests. In most cases, these are unit tests or integration tests that check the interfaces of the subsystem you want to improve. If the tests still run successfully after the change, this is a first indication of successful refactoring. However, if the tests fail, you must reverse the change, check it, or at least analyze why the tests failed.

Automated tests don't replace a manual check of the refactoring result or a manual test, but they do give you security and relieve you of some of the manual testing effort.

4.3.4 Defining Clear Goals

Especially when refactoring using AI tools, there is a great risk that things will get out of hand and you'll rebuild too much. Before you start adapting the source code, you should set yourself a specific goal, such as improving the readability of the source code or introducing certain structural improvements. Prior to each refactoring step, you should check whether the desired modification will move you closer to your goal. If that isn't the case, you shouldn't do it.

In this chapter, you've seen that AI tools such as GitHub Copilot provide fast and comprehensive refactoring procedures. To really work in small steps and in a goal-oriented manner, you should limit the result with prompts that are as precise as possible.